

Python OOP Extra Notes + Detailed Practice Sheet (English Only)

These notes cover important OOP topics that may be *outside* the 4 intro PDFs (Concepts, Constructor, Inheritance, Polymorphism).
Use this as a compact study guide **and** a problem set.

Part A — Notes (Detailed)

1) Encapsulation (in depth)

Definition: Encapsulation means bundling data (attributes) and behavior (methods) inside a class and controlling how that data is accessed or changed from outside.

Why it matters:

- Protects object state from accidental or invalid updates.
- Makes code easier to maintain because invariants are enforced in one place.

Encapsulation tools in Python - Public attribute: `x` → can be accessed/modified anywhere. - **Protected convention:** `_x` → should be treated as internal (still accessible). - **Private convention:** `__x` → triggers *name mangling* to reduce accidental access. - **Managed access:** `@property` with setter/getter.

Example

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # "private" by name mangling

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount < 0:
            raise ValueError("Balance cannot be negative")
        self.__balance = amount
```

Common mistakes & likely errors 1. **Accessing a private attribute directly** python `acc = BankAccount(100) print(acc.__balance)`
Error: `AttributeError: 'BankAccount' object has no attribute '__balance'`

Because Python renamed it to `_BankAccount__balance`.

2. Infinite recursion in a setter

```

@balance.setter
def balance(self, amount):
    self.balance = amount    # BAD

```

Error: RecursionError: maximum recursion depth exceeded
 Setter calls itself repeatedly. Use a hidden attribute instead
 (self.__balance).

3. Using @property without defining a setter but trying to assign

```

class A:
    @property
    def x(self): return 5

```

```

a = A()
a.x = 10

```

Error: AttributeError: can't set attribute
 Fix by adding @x.setter.

2) Abstraction (in depth)

Definition: Abstraction means exposing *what an object does* while hiding *how it does it*.

In Python, abstraction is commonly implemented using **Abstract Base Classes (ABCs)**.

Why it matters - Forces consistent interfaces across subclasses. - Prevents incomplete class definitions.

Tools: abc module, ABC, @abstractmethod

Example

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h

```

Common mistakes & likely errors 1. Instantiating an abstract class
python `s = Shape()` **Error:** `TypeError: Can't instantiate abstract class Shape with abstract methods area`

2. Forgetting to implement an abstract method in child

```
class Circle(Shape):  
    pass
```

```
Circle()
```

Error: Same `TypeError` as above.

3. Misspelling method name If ABC declares `area()` but child defines `Area()`:

- No override happens.
 - Instantiation fails with abstract-method error.
-

3) Multiple Inheritance & MRO

Multiple inheritance: A class inherits from more than one parent.

MRO (Method Resolution Order): The order Python uses to search for methods/attributes.

Example

```
class A:  
    def f(self): print("A")  
  
class B:  
    def f(self): print("B")  
  
class C(A, B):  
    pass  
  
c = C()  
c.f()           # prints "A" because A is first  
print(C.mro()) # shows lookup order
```

Common mistakes & likely errors 1. Conflicting method signatures

If parents define `f(self, x)` and `f(self, y, z)`, calling child `f()` may raise:
Error: `TypeError: f() missing required positional arguments`

2. Not using super() correctly in diamond inheritance Forgetting `super()` can cause a parent constructor to run twice or not at all.

4) Magic (Dunder) Methods & Operator Overloading

Definition: Dunder methods let your objects behave like built-in types.

Examples:

- `print(obj)` → `__str__` or `__repr__`
- `len(obj)` → `__len__`
- `obj1 + obj2` → `__add__`

Example

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"
```

Common mistakes & likely errors 1. Adding different types python

```
v = Vector(1,2)    print(v + 5)    Error:      AttributeError: 'int'
object has no attribute 'x'
```

Fix: type-check other inside `__add__`.

2. Returning non-int from `__len__`

```
def __len__(self): return "5"
```

Error: TypeError: 'str' object cannot be interpreted as an integer

5) Composition vs Inheritance

- Inheritance (“is-a”): Student is a Person.
- Composition (“has-a”): Car has an Engine.

Why composition is often preferred - Less coupling (changes in one class don't break others). - More flexible: you can swap components.

Example

```
class Engine:
    def start(self):
        print("Engine on")

class Car:
    def __init__(self):
```

```

        self.engine = Engine()

    def drive(self):
        self.engine.start()
        print("Driving")

```

Common mistakes & likely errors - Forgetting to initialize composed object:

```
python class Car:
    def drive(self):
        self.engine.start()
Error: AttributeError: 'Car' object has no attribute 'engine'
```

6) @classmethod and @staticmethod

staticmethod - Belongs to class namespace but doesn't access instance (**self**) or class (**cls**).

- Good for utility functions.

classmethod - Receives **cls** and can modify class state or create alternative constructors.

Example

```

class Person:
    count = 0

    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def from_fullname(cls, fullname):
        return cls(fullname.strip())

    @staticmethod
    def is_adult(age):
        return age >= 18

```

Common mistakes & likely errors 1. Using **self** inside **staticmethod**

```
python @staticmethod def f():
    return self.x
Error:
NameError: name 'self' is not defined
```

2. Calling **classmethod** on instance but expecting instance fields **cls** only sees class-level fields, not per-object attributes.
-

7) Name Mangling (Private Convention)

`__x` becomes `_ClassName__x`.

Purpose: avoid accidental access, *not* full security.

Common mistake - Accessing `__x` directly → `AttributeError` (as in Encapsulation section).

8) Custom Exceptions

Definition: User-defined errors for clear domain-specific failure messages.

Example

```
class InsufficientBalanceError(Exception):
    pass

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError("Not enough money")
        self.balance -= amount
```

Common mistakes & likely errors - Raising a string instead of Exception:
python `raise "error"` **Error:** `TypeError: exceptions must derive from BaseException`

9) `__new__` vs `__init__`

- `__new__(cls, ...)` creates the object (rarely overridden).
- `__init__(self, ...)` initializes it.

When you care - Immutables, caching, Singleton.

Common errors - Returning `None` from `__new__` → object not created → later `AttributeErrors`.

10) SOLID Principles (quick scope)

- **S** Single Responsibility: one class, one reason to change.

- **O** Open/Closed: open for extension, closed for modification.
 - **L** Liskov Substitution: child should replace parent safely.
 - **I** Interface Segregation: many small interfaces are better than one fat one.
 - **D** Dependency Inversion: depend on abstractions, not details.
-

Part B — Practice Sheet (Beginner → Mid)

Section 1: Encapsulation & Properties

1. Create `Student` with private `__cgpa`.
 - Use `cgpa` property getter + setter.
 - If `cgpa` not in `[0, 4]`, raise `ValueError`.
 2. Create `Temperature` with private `__celsius`.
 - Read-only property `fahrenheit` returns converted value.
-

Section 2: Abstraction

3. Create abstract class `Vehicle` with abstract method `max_speed()`.
 - Implement in `Car`, `Bike`.
 4. Create abstract class `PaymentMethod` with `pay(amount)`.
 - Implement in `Bkash`, `Card` to print payment confirmation.
-

Section 3: Multiple Inheritance & MRO

5. Create `Writer.work()` and `Singer.work()`.
 - Create `Artist(Writer, Singer)`.
 - Call `work()` and print `Artist.mro()`.
 6. Create diamond inheritance `A -> B, A -> C, D(B, C)`.
 - Use `super()` in constructors and show correct call order.
-

Section 4: Magic Methods

7. Create `ComplexNo`:
 - fields: `real`, `imag`
 - overload `+` using `__add__`
 - pretty print using `__str__`
8. Create `Book`:
 - fields: `title`, `pages`

- implement `__len__` so `len(book)` returns pages.
-

Section 5: Composition vs Inheritance

9. Create `Library` that *has* a list of `Book`.
 - `add_book(book)`
 - `total_books()`
 10. Create `Printer` and `Computer` (`Computer` *has* `Printer`).
 - `Computer.print_doc(text)` delegates to `printer`.
-

Section 6: Classmethod / Staticmethod

11. `Employee`:
 - class var `tax_rate`
 - classmethod `set_tax_rate(r)`
 - instance method `salary_after_tax()`
 12. staticmethod `is_valid_email(email)`:
 - returns `True` if contains exactly one `@` and a dot after it.
-

Section 7: Custom Exceptions

13. Make `InsufficientBalanceError`.
 - Use inside `BankAccount.withdraw(amount)`.
-

Part C — Mini Project (Highly Recommended)

Project: Simple Banking System

Requirements - `BankAccount` - private `balance` - `deposit`, `withdraw` (raise custom exception when needed) - `__str__` shows account summary - `SavingsAccount(BankAccount)` - `interest_rate` - `add_interest()` updates `balance` - Main program: - create 3 accounts - demonstrate `deposit`, `withdraw`, `interest`, exception handling

Extra credit - Add transaction history list. - Use `@property` to guard `balance` access.

How to Submit for Checking

Send: - `.py` file(s) or code blocks per question - output screenshots are optional

I will review logic, OOP design, and point out improvements.